

Fakultet tehničkih nauka
Univerzitet u Novom Sadu

Paralelne i distribuirane arhitekture i jezici

Osnovni koncepti

Branislav Ristić

Zimski semestar, školska 2026./27.

Promenljive i konstante

Simboličke adrese

- Vrednosti podrazumevano nepromenljive.
- Definicija i inicijalizacija:

```
let name: type = value
```

- Primer:

primeri/basic-concepts/src/bin/standard.rs

```
fn main() {
    let i: i32 = 5;
    println!("The value of i is: {}", i);

    let c: char = 'z';
    println!("The value of c is: {}", c);

    let f: f64 = 6.4;
    println!("The value of f is: {}", f);

    let b: bool = true;
    println!("The value of b is: {}", b);
}
```

Izlaz

```
The value of i is: 5
The value of c is: z
The value of f is: 6.4
The value of b is: true
```

Promenljive

- Simboličke adrese čije su vrednosti promenljive.
- Definicija i inicijalizacija:

```
let mut name: type = value
```

- Primer:

primeri/basic-concepts/src/bin/variables.rs

```
fn main() {
    let mut i: i32 = 5;
    println!("The value of i is: {}", i);
    i = 6;
    println!("The value of i is: {}", i);
}
```

Izlaz

```
The value of i is: 5
The value of i is: 6
```

Konstante

- Identifikatori kojima se dodeljena vrednost ne može menjati.

- Definisanje i inicijalizacija:

```
const NAME: type = value
```

- Primer:

primeri/basic-concepts/src/bin/constants.rs

```
fn main() {
    const I: i32 = 5;
    println!("The value of I is: {}", I);

    const C: char = 'z';
    println!("The value of C is: {}", C);

    const F: f64 = 6.4;
    println!("The value of F is: {}", F);

    const B: bool = true;
    println!("The value of B is: {}", B);
}
```

Izlaz

```
The value of I is: 5
The value of C is: z
The value of F is: 6.4
The value of B is: true
```

Pitanje

- Kada koristiti:
  - Nepromenljive simboličke adrese?
  - Konstante?

`const` vs. `let`

Osobina	`const`	`let`
Promenljivost	Uvek nepromenljiva	Nepromenljiva podrazumevano, može `mut`
Deklaracija	`const`	`let`
Tip	Obavezan	Opcion (izvodi se)
Evaluiranje	U vreme kompajliranja	U vreme izvršavanja
Memorija	Ugrađena u kod (bez memorije)	Čuva se u memoriji
Opseg	Globalni ili na nivou modula	Blok (lokalni)
Izraz	Mora biti konstantan	Bilo koji izraz

Shadowing

- Dozvoljeno je definisanje simboličkih adresa sa istim imenom više puta.
- Uvek je važeća simbolička adresa koja je poslednja deklarisana.
- Važi:
  - Dok se ne deklariše nova sa istim identifikatorom.
  - Dok ne izađe iz opsega u kom je definisana.

Shadowing: novo vezivanje, opseg, promena tipa

primeri/basic-concepts/src/bin/shadowing.rs

```rust
fn main() {
    let x = 5;
    println!("x: {}, address: {:p}", x, &x);

    let x = x + 1;
    println!("x before again in scope: {}, address: {:p}", x, &x);

    {
        let x = 3;
        println!("x: {}, address: {:p}", x, &x);
        {
            let x = "hello";
            println!("x: {}, address: {:p}", x, &x);
        }
    }
    println!("x after again in scope: {}, address: {:p}", x, &x);

    let x = x * 2;
    println!("x: {}, address: {:p}", x, &x);

    let x = "world";
    println!("x: {}, address: {:p}", x, &x);

    let mut x = 42;
    println!("x: {}, address: {:p}", x, &x);

    // x = "fail";
    // println!("x: {}, address: {:p}", x, &x);
}
```

Izlaz

```
x: 5, address: 0x7ffd26ef7aa4
x before again in scope: 6, address: 0x7ffd26ef7af4
x: 3, address: 0x7ffd26ef7b44
x: hello, address: 0x7ffd26ef7b90
x after again in scope: 6, address: 0x7ffd26ef7af4
x: 12, address: 0x7ffd26ef7c34
x: world, address: 0x7ffd26ef7c80
x: 42, address: 0x7ffd26ef7cdc
```

Tipovi podataka

Tipovi podataka u Rust-u

Kategorija	Tip	Primer
Skalarni	`i8` ... `i128`, `isize`	`let x: i32 = 10;`
	`u8` ... `u128`, `usize`	`let y: u64 = 100;`
	`f32`, `f64`	`let z: f64 = 6.4;`
	`bool`	`let flag: bool = true;`
	`char`	`let letter: char = 'a';`
Složeni	`tuple`	`let tup = (1, 2.0, 'c');`
	`array`	`let arr = [1, 2, 3];`
Jedinični	`()`	`let unit = ();`

Kategorija	Tip	Primer

Operacije nad skalarnim tipovima u Rust-u

Tip	Operacije
Celi brojevi	+, -, *, /, %, ==, !=, <, >, <=, >=
Realni brojevi	Iste kao za cele brojeve
`bool`	&&, ||, !
`char`	==, !=, <, >, <=, >=

Brojevi: metode, granice tipa, prekoračenje

- Celobrojno deljenje odseca decimale; `%` daje ostatak.
- Prekoračenje običnog +: u debug build-u program izaziva paniku, a u release build-u se (podrazumevano) vrednost obmotava; oslanjati se na to je greška.
- `checked_*` vraća `Option`, a `wrapping_*` obmotava vrednost.
- Literali: `1_000_000`, `0xff`, `0o77`, `0b1010`, sufiks tipa `10_u8`.

Brojevi: primer

primeri/basic-concepts/src/bin/numbers.rs

```rust
fn main() {
    // Number literals: separators, bases and type suffixes
    println!("{} {} {} {}", 1_000_000, 0xff, 0o77, 0b1010);
    println!("{} {}", 10_u8, 2.5_f32);

    // Integer division truncates, % gives the remainder
    println!("7 / 2 = {}, 7 % 2 = {}", 7 / 2, 7 % 2);
    println!("7.0 / 2.0 = {}", 7.0 / 2.0);

    // Methods on numbers
    println!("abs: {}, pow: {}", (-5_i32).abs(), 2_i32.pow(10));
    println!("sqrt: {}, round: {}", 16.0_f64.sqrt(), 2.567_f64.round());
    println!("min: {}, max: {}", 3_i32.min(8), 3_i32.max(8));

    // Limits of a type
    println!("i32::MAX = {}", i32::MAX);

    // Overflow: checked_add returns None instead of panicking
    println!("checked_add: {:?}", i32::MAX.checked_add(1));
    println!("wrapping_add: {}", i32::MAX.wrapping_add(1));
}
```

Izlaz

```
1000000 255 63 10
10 2.5
7 / 2 = 3, 7 % 2 = 1
7.0 / 2.0 = 3.5
abs: 5, pow: 1024
sqrt: 4, round: 3
min: 3, max: 8
i32::MAX = 2147483647
checked_add: None
wrapping_add: -2147483648
```

Ispis: formatiranje sa `println!`

Oznaka	Značenje	Primer
`{}`	Display (ispis korisniku)	`println!("{}", 5)`
`{:?}`	Debug (nizovi, vektori, torke)	`println!("{:?}", [1, 2])`
`{:.2}`	Broj decimala	`println!("{:.2}", 3.14159)`
`{:>5}`	Poravnanje na širinu 5	`println!("{:>5}", 42)`

- `!` na kraju imena znači da je u pitanju makro, a ne funkcija (`println!`, `vec!`, `panic!`).
- Promenljiva se može napisati direktno u zagradama: `{n}`, `{pi:.1}` (samo imena, ne izrazi).

Ispis: Display, Debug, decimale, poravnanje

`primeri/basic-concepts/src/bin/formatting.rs`

```rust
fn main() {
    let n = 42;
    let pi = 3.14159;
    let array = [1, 2, 3];
    let tuple = (1, "two", 3.0);

    // Display: {}
    println!("{}", n);

    // Debug: {:?} (arrays and tuples have no Display)
    println!("{:?} {:?}", array, tuple);

    // Number of decimals
    println!("{:.2}", pi);

    // Width and alignment: right, left, center
    println!("[{:>5}] [{:<5}] [{:^5}]", n, n, n);

    // Variables can be written directly inside the braces
    println!("n = {n}, pi = {pi:.1}");

    // `vec!` is also a macro (the `!` in the name)
    let v = vec![1, 2, 3];
    println!("{:?}", v);
}
```

`Izlaz`

```
42
[1, 2, 3] (1, "two", 3.0)
3.14
[   42] [42   ] [ 42  ]
n = 42, pi = 3.1
[1, 2, 3]
```

stdin

- Primer:

`primeri/basic-concepts/src/bin/stdin.rs`

```rust
use std::io;

fn main() {
    println!("{}", String::from("Type something:"));
    let mut input = String::new();
    io::stdin().read_line(&mut input).unwrap();
    println!("You typed: {}", input);
}
```

Izlaz

```
Type something:
hello
You typed: hello
```

Type casting – as

- Ključna reč `as` omogućava eksplicitno pretvaranje vrednosti između skalara, odnosno prostih tipova u Rust-u.
- Rust ne vrši implicitnu konverziju između brojčanih tipova.
- Pravila:
  - *Truncation*: ceo broj u manji ceo broj zadržava niže bitove, a realan broj se odseca ka nuli (`-42.9 as i32` je -42),
  - *Saturation*: realan broj van opsega postaje najbliža granica, NaN postaje 0; isti rezultat u debug i release build-u.
- `to_int_unchecked` (`unsafe`) preskače proveru: van opsega je ponašanje nedefinisano i rezultat zavisi od build-a.

Type casting – as: odsecanje i zasićenje

primeri/basic-concepts/src/bin/casting_as.rs

```rust
// Suppress all warnings from casts which overflow.
#![allow(overflowing_literals)]

fn main() {
    let decimal = 65.4321_f32;

    // Error! No implicit conversion
    // let integer: u8 = decimal;

    // Explicit conversion
    let integer = decimal as u8;
    let character = integer as char;
    println!("Casting: {} -> {} -> {}", decimal, integer, character);

    // Error! There are limitations in conversion rules.
    // A float cannot be directly converted to a char.
    // let character = decimal as char;

    // 1000 already fits in a u16
    println!("1000 as a u16 is: {}", 1000 as u16);

    // 1000 - 256 - 256 - 256 = 232
    // Under the hood, the first 8 least significant bits (LSB) are kept,
    // while the rest towards the most significant bit (MSB) get
     ↪  truncated.
    println!("1000 as a u8 is : {}", 1000 as u8);
```

```
    // -1 + 256 = 255
    println!("  -1 as a u8 is : {}", (-1i8) as u8);

    // For positive numbers, this is the same as the modulus
    println!("1000 mod 256 is : {}", 1000 % 256);

    // When casting to a signed type, the (bitwise) result is the same as
    // first casting to the corresponding unsigned type. If the most
     ↪  significant
    // bit of that value is 1, then the value is negative.

    // Unless it already fits, of course.
    println!(" 128 as a i16 is: {}", 128 as i16);

    // In boundary case 128 value in 8-bit two's complement representation
     ↪  is -128
    println!(" 128 as a i8 is : {}", 128 as i8);

    // repeating the example above
    // 1000 as u8 -> 232
    println!("1000 as a u8 is : {}", 1000 as u8);
    // and the value of 232 in 8-bit two's complement representation is
     ↪  -24
    // 232 = 0b11101000 -> -128 + 64 + 32 + 8 = -24
    println!(" 232 as a i8 is : {}", 232 as i8);

    // Since Rust 1.45, the `as` keyword performs a *saturating cast*
    // when casting from float to int. If the floating point value exceeds
    // the upper bound or is less than the lower bound, the returned value
    // will be equal to the bound crossed.
    // The result is the same in debug and release builds.

    // 300.0 as u8 is 255
    println!(" 300.0 as u8 is : {}", 300.0_f32 as u8);
    // -100.0 as u8 is 0
    println!("-100.0 as u8 is : {}", -100.0_f32 as u8);
    // nan as u8 is 0
    println!("   nan as u8 is : {}", f32::NAN as u8);

    // The saturating check incurs a small runtime cost and can be skipped
    // with the unsafe `to_int_unchecked`. If the value is out of range or
    // NaN, the behavior is undefined: the result below depends on the
    // build (debug prints 44 and 156, release prints 0 and 0).
    unsafe {
        println!("--- Unsafe casting methods ---");
        println!(" 300.0 as u8 is : {}",
         ↪  300.0_f32.to_int_unchecked::<u8>());
        println!(
            "-100.0 as u8 is : {}",
            (-100.0_f32).to_int_unchecked::<u8>()
        );
        // nan is also undefined; it only happens to print 0 here
        println!("   nan as u8 is : {}",
         ↪  f32::NAN.to_int_unchecked::<u8>());
    }
}
```

Izlaz

```
Casting: 65.4321 -> 65 -> A
1000 as a u16 is: 1000
1000 as a u8 is : 232
  -1 as a u8 is : 255
1000 mod 256 is : 232
 128 as a i16 is: 128
 128 as a i8 is : -128
1000 as a u8 is : 232
 232 as a i8 is : -24
 300.0 as u8 is : 255
-100.0 as u8 is : 0
   nan as u8 is : 0
--- Unsafe casting methods ---
 300.0 as u8 is : 44
-100.0 as u8 is : 156
   nan as u8 is : 0
```

Type casting – (Try)From/(Try)Into

- `From/Into` osobine definišu metode za konverzije koje se uvek mogu izvršiti bez greške.
- `TryFrom/TryInto` za konverzije koje se ne mogu uvek izvršiti bez greške.
- Definiše se samo `From`, pa se recipročno dobija `Into`.

Type casting – From/Into: sa greškom i bez nje

primeri/basic-concepts/src/bin/casting_from_into.rs

```rust
use std::convert::{From, TryFrom, TryInto};

fn main() {
    // Example 1: Using From / Into for standard library types
    {
        // From<&str> for String is provided by std
        let s: &str = "hello";
        let string1 = String::from(s);
        let string2: String = s.into(); // Into<String> works because
         ↪  From<&str> for String is implemented
        println!("string1 = {}, string2 = {}", string1, string2);
    }

    // Example 2: Primitive conversions between numeric types via
     ↪  From/Into
    {
        // many primitive conversions in std are implemented
        let x_u8: u8 = 100;
        let x_u16: u16 = x_u8.into(); // u8 -> u16: widening, infallible
        let x_u16: u16 = u16::from(x_u8); // equivalent using From
        println!("u8 -> u16: {} -> {}", x_u8, x_u16);

        let x_i32: i32 = 5000;
        let x_i64: i64 = x_i32.into(); // widening signed int
        let x_i64: i64 = i64::from(x_i32); // equivalent using From
        println!("i32 -> i64: {} -> {}", x_i32, x_i64);

        // Now we try to convert from a larger to a smaller type
        // let y_i16: i16 = 300;
        // let y_i8: i8 = y_i16.into(); // This line would not compile: no
         ↪  From<i16> for i8
```

```
    }

    // Example 3: Fallible conversion with TryFrom / TryInto
    {
        // Suppose we want to convert i64 -> u8, but check for overflow
        let big: i64 = 1000;
        // This conversion may fail: 1000 > u8::MAX (255)
        let maybe_small: Result<u8, _> = u8::try_from(big);
        match maybe_small {
            Ok(v) => println!("Converted: {}", v),
            Err(e) => println!("Conversion failed: {:?} (value out of
             ↪  range)", e),
        }

        let small: i64 = 200;
        let maybe_small2: Result<u8, _> = small.try_into();
        println!("i64({}) -> u8 via try_into(): {:?}", small,
         ↪  maybe_small2);
    }
}
```

Izlaz

```
string1 = hello, string2 = hello
u8 -> u16: 100 -> 100
i32 -> i64: 5000 -> 5000
Conversion failed: TryFromIntError(PosOverflow) (value out of range)
i64(200) -> u8 via try_into(): Ok(200)
```

Type casting – parse

- Metoda `parse` pretvara tekst (`&str`) u broj ili drugi tip.
- Pretvaranje može da ne uspe, pa se vraća `Result`.
- Razmaci i novi red na početku ili kraju teksta su greška (zato se pri čitanju sa standardnog ulaza koristi `trim`).

Type casting – parse: uspeh i greška

primeri/basic-concepts/src/bin/string_parse.rs

```
fn main() {
    let good = "123";
    let bad1 = "42a";
    let bad2 = "   456\n"; // contains whitespace/newline

    // Example 1: parse success
    let n: i32 = good.parse().unwrap();
    println!("Parsed '{}': {}", good, n);

    // Example 2: parse invalid string -> Err
    match bad1.parse::<i32>() {
        Ok(v) => println!("Parsed '{}': {}", bad1, v),
        Err(e) => println!("Failed to parse '{}': {}", bad1, e),
    }

    // Example 3: parse with whitespace/newline -> Err
    match bad2.parse::<i32>() {
        Ok(v) => println!("Parsed '{}': {}", bad2, v),
```

```
        Err(e) => println!("Failed to parse '{}': {}", bad2, e),
    }
}
```

Izlaz

```
Parsed '123': 123
Failed to parse '42a': invalid digit found in string
Failed to parse '   456
': invalid digit found in string
```

Složeni tipovi

Tuple

- Torka.
- Grupisanje više vrednosti različitih tipova u jedan složen tip.
- Dužina i tipovi su fiksni; vrednosti se mogu menjati ako je torka `mut`.
- Definicija:

```
let tup: (type1, type2, ...) = (value1, value2, ...)
```

Tuple: destrukturiranje i pristup po indeksu

primeri/basic-concepts/src/bin/tuple.rs

```
fn main() {
    let tup: (i32, f64, u8) = (500, 6.4, 1);

    let (x, y, z) = tup;
    println!("The values are: x = {}, y = {}, z = {}", x, y, z);
    println!(
        "The values are: tup.0 = {}, tup.1 = {}, tup.2 = {}",
        tup.0, tup.1, tup.2
    );

    let mut tup: (i32, f64, u8) = (500, 6.4, 1);
    tup.0 = 600;
    println!("The updated value of tup.0 is: {}", tup.0);
}
```

Izlaz

```
The values are: x = 500, y = 6.4, z = 1
The values are: tup.0 = 500, tup.1 = 6.4, tup.2 = 1
The updated value of tup.0 is: 600
```

Array

- Niz elemenata istog tipa.
- Fiksne dužine.
- Definicija i inicijalizacija:

```
let name: [type; no_elems] = [el1, el2, ...];
let name = [elem; no_elems];
```

Array: pristup elementu po indeksu

primeri/basic-concepts/src/bin/array.rs

```rust
fn main() {
    let a = [1, 2, 3, 4, 5];
    println!("a: {:?}", a);

    let first = a[0];
    println!("first: {}", first);

    let second = a[1];
    println!("second: {}", second);

    println!("length: {}", a.len());

    // Array with the same value repeated
    let zeros = [0; 5];
    println!("zeros: {:?}", zeros);
}
```

Izlaz

```
a: [1, 2, 3, 4, 5]
first: 1
second: 2
length: 5
zeros: [0, 0, 0, 0, 0]
```

Array: panika zbog indeksa van opsega

primeri/basic-concepts/src/bin/stdin_index.rs

```rust
use std::io;

fn main() {
    let array1 = [10, 20, 30, 40, 50];
    println!("Array array1: {:?}", array1);

    println!(
        "{}",
        String::from("Enter index [0-4], if not valid, rust will panic:")
    );
    let mut input = String::new();
    io::stdin().read_line(&mut input).unwrap();

    let index: usize = input.trim().parse::<usize>().expect("Not a valid
    ↪ number");
    let value_indexed = array1[index];
    println!("Value at index {} is {}", index, value_indexed);
}
```

Izlaz

```
Array array1: [10, 20, 30, 40, 50]
Enter index [0-4], if not valid, rust will panic:
7

thread 'main' (5621) panicked at
↪ basic-concepts/src/bin/stdin_index.rs:15:25:
```

```
index out of bounds: the len is 5 but the index is 7
note: run with `RUST_BACKTRACE=1` environment variable to display a
 ↪  backtrace
```

Array – traits

- `Copy`.
- `Clone`.
- `Debug`.
- `IntoIterator` (implementirano za `[T; N]`, `&[T; N]` i `&mut [T; N]`).
- `PartialEq`, `PartialOrd`, `Eq`, `Ord`.
- `Hash`.
- `AsRef`, `AsMut`.
- `Borrow`, `BorrowMut`.

Array – traits: kopiranje, poređenje, iteracija

primeri/basic-concepts/src/bin/array_traits.rs

```rust
use std::borrow::Borrow;
use std::borrow::BorrowMut;
use std::cmp::Ordering;
use std::collections::hash_map::DefaultHasher;
use std::hash::{Hash, Hasher};

fn main() {
    // Array demonstrating Copy and Clone
    let mut array1: [i32; 3] = [1, 2, 3];
    let array2 = array1; // Copy occurs here, because [i32; 3] is Copy
     ↪  (i32 is Copy)
    let array3 = array1.clone(); // Clone occurs here

    println!("Array 1: {:?}", array1);
    println!("Array 2 (Copy of Array 1): {:?}", array2);
    println!("Array 3 (Clone of Array 1): {:?}", array3);

    let array4: [i32; 3] = [2; 3];
    let array5: [i32; 3] = [1, 2, 3];

    println!("Array 4 == Array 5: {}", array4 == array5); // PartialEq
    println!("Array 4 > Array 5: {}", array4 > array5); // PartialOrd (cmp
     ↪  below uses Ord)

    match array4.cmp(&array5) {
        Ordering::Less => println!("Array 4 is less than Array 5"),
        Ordering::Equal => println!("Array 4 is equal to Array 5"),
        Ordering::Greater => println!("Array 4 is greater than Array 5"),
    }

    let array8: [f64; 3] = [1.0, 2.0, 3.0];
    let array9: [f64; 3] = [1.0, 2.0, 4.0];

    // Does not compile: the length is part of the type
    // let array8: [f64; 3] = [1.0, 2.0, 3.0, 4.0];

    println!("Array 8 == Array 9: {}", array8 == array9); // PartialEq
    println!("Array 8 > Array 9: {}", array8 > array9); // PartialOrd
```

```
    match array8.partial_cmp(&array9) {
        Some(Ordering::Less) => println!("Array 8 is less than Array 9"),
        Some(Ordering::Equal) => println!("Array 8 is equal to Array 9"),
        Some(Ordering::Greater) => println!("Array 8 is greater than Array
         ↪  9"),
        None => println!("Array 8 and Array 9 are not comparable"),
    }

    // Demonstrating Hash (DefaultHasher output is not stable across Rust
     releases)
    let mut hasher = DefaultHasher::new();
    array1.hash(&mut hasher);
    let hash_value = hasher.finish();
    println!("Hash of Array 1: {}", hash_value);

    // Demonstrating AsRef and AsMut
    // Used predominantly for converting to references of other types
    let array6: [i32; 3] = [10, 20, 30];
    let slice_ref: &[i32] = array6.as_ref(); // AsRef
    println!("Array 6 as slice (AsRef): {:?}", slice_ref);

    let mut array7 = [5, 6, 7];
    let slice_mut: &mut [i32] = array7.as_mut(); // AsMut
    println!("Array 7 as mutable slice (AsMut): {:?}", slice_mut);
    slice_mut[0] = 50;
    println!("Array 7 as mutable slice (AsMut): {:?}", slice_mut);

    // Demonstrating Borrow and BorrowMut
    // Used predominantly for looking up values in collections (like
     ↪  HashMap) by converting to references (String to &str)
    let borrowed_slice: &[i32] = array6.borrow(); // Borrow
    println!("Borrowed slice (Borrow): {:?}", borrowed_slice);

    let borrowed_mut_slice: &mut [i32] = array7.borrow_mut(); // BorrowMut
    println!(
        "Borrowed mutable slice (BorrowMut): {:?}",
        borrowed_mut_slice
    );
    borrowed_mut_slice[0] = 100;
    println!(
        "Borrowed mutable slice (BorrowMut): {:?}",
        borrowed_mut_slice
    );

    // Demonstrating IntoIterator by value (array1 stays usable, [i32; 3]
     ↪  is Copy)
    println!("Iterating over Array 1 using IntoIterator:");
    for elem in array1 {
        println!("{}", elem);
    }

    // Iterating over a reference to an array (IntoIterator for &[i32; 3])
    println!("Iterating over Array 1 by reference:");
    for elem in &array1 {
        println!("{}", elem);
    }
```

```
    // Iterating over a mutable reference to an array (IntoIterator for
     ↪  &mut [i32; 3])
    println!("Iterating over Array 1 by mutable reference:");
    for elem in &mut array1 {
        println!("{}", elem);
    }

    // Debug trait is demonstrated through all the prints
    println!("Array 1: {:?}", array1);
    println!("Array 2 (Copy of Array 1): {:?}", array2);
    println!("Array 3 (Clone of Array 1): {:?}", array3);
    println!("Array 4: {:?}", array4);
}
```

Izlaz

```
Array 1: [1, 2, 3]
Array 2 (Copy of Array 1): [1, 2, 3]
Array 3 (Clone of Array 1): [1, 2, 3]
Array 4 == Array 5: false
Array 4 > Array 5: true
Array 4 is greater than Array 5
Array 8 == Array 9: false
Array 8 > Array 9: false
Array 8 is less than Array 9
Hash of Array 1: 13585085576907263210
Array 6 as slice (AsRef): [10, 20, 30]
Array 7 as mutable slice (AsMut): [5, 6, 7]
Array 7 as mutable slice (AsMut): [50, 6, 7]
Borrowed slice (Borrow): [10, 20, 30]
Borrowed mutable slice (BorrowMut): [50, 6, 7]
Borrowed mutable slice (BorrowMut): [100, 6, 7]
Iterating over Array 1 using IntoIterator:
1
2
3
Iterating over Array 1 by reference:
1
2
3
Iterating over Array 1 by mutable reference:
1
2
3
Array 1: [1, 2, 3]
Array 2 (Copy of Array 1): [1, 2, 3]
Array 3 (Clone of Array 1): [1, 2, 3]
Array 4: [2, 2, 2]
```

Funkcije i kontrola toka

Funkcije

- Unutar svakog binarnog crate-a postoji `main` funkcija:

```
fn naziv_funkcije([param1, param2, ...]) -> tip {
    Statements;
}
```

- Parametri su podrazumevano nepromenljivi; `mut` ispred parametra menja samo lokalnu kopiju. Da bi se promenila vrednost pozivaoca, koristi se referenca `&mut` (tema Vlasništvo).
- Više vrednosti se vraća kao torka (*tuple*), npr. `-> (i32, i32)`.

Funkcije: parametri, povratna vrednost, rekurzija

primeri/basic-concepts/src/bin/functions.rs

```
// Basic function declaration with no return value (unit type `()` is
 ↪  implied)
fn say_hello() {
    println!("Hello, world!");
}

// Function with parameters and an explicit return type (i32)
fn add(a: i32, b: i32) -> i32 {
    a + b
}

// Function with an early return using `return` keyword (optional, as Rust
 ↪  returns the last expression by default)
fn is_even(n: i32) -> bool {
    if n % 2 == 0 {
        return true;
    }
    false
}

// Recursive function: calculates factorial of a number
fn factorial(n: u32) -> u32 {
    if n <= 1 {
        1
    } else {
        n * factorial(n - 1)
    }
}

// Returning several values: a tuple as the return type
fn sort_pair(a: i32, b: i32) -> (i32, i32) {
    if a < b {
        (a, b)
    } else {
        (b, a)
    }
}

// `mut` on a parameter makes only the local copy mutable;
// the caller's variable is not changed
fn add_one(mut x: i32) -> i32 {
    x += 1;
    x
}

// Function with an optional argument: Rust has no default parameters, so
 ↪  `Option` is used instead
fn greet(name: Option<&str>) {
    match name {
        Some(n) => println!("Hello, {}!", n),
```

```
        None => println!("Hello, stranger!"),
    }
}

fn main() {
    // Call the basic function
    say_hello();

    // Call the add function
    let sum = add(5, 7);
    println!("Sum of 5 and 7: {}", sum);

    // Check if a number is even
    let number = 10;
    println!("Is {} even? {}", number, is_even(number));

    // Call the recursive factorial function
    let fact_of_5 = factorial(5);
    println!("Factorial of 5: {}", fact_of_5);

    // Call the function with a `mut` parameter
    let n = 5;
    let m = add_one(n);
    println!("n = {}, add_one(n) = {}", n, m);

    // Destructure the returned tuple
    let (low, high) = sort_pair(8, 3);
    println!("low = {}, high = {}", low, high);

    // Call function with optional arguments
    greet(Some("Alice"));
    greet(None);
}
```

Izlaz

```
Hello, world!
Sum of 5 and 7: 12
Is 10 even? true
Factorial of 5: 120
n = 5, add_one(n) = 6
low = 3, high = 8
Hello, Alice!
Hello, stranger!
```

Iskaz i izraz (statement and expression)

- *Statement* (iskaz):
  - Instrukcije koje izvršavaju neku radnju i ne vraćaju vrednost.
    - Kreiranje promenljive i dodeljivanje vrednosti pomoću `let`.
    - Definisanje funkcije.
- *Expression* (izraz):
  - Izračunava se do rezultujuće vrednosti.

Iskaz i izraz: blok kao izraz

`primeri/basic-concepts/src/bin/expressions.rs`

```rust
fn main() {
    // A block is an expression: its last expression (no `;`) is the value
    let y = {
        let x = 3;
        x + 1
    };
    println!("y = {}", y);

    // With `;` the block would return () instead of a value
    let unit = {
        let x = 3;
        x + 1;
    };
    println!("unit = {:?}", unit);

    // if is an expression too
    let size = if y > 3 { "big" } else { "small" };
    println!("size = {}", size);
}
```

Izlaz

```
y = 4
unit = ()
size = big
```

Selekcija – IF

- ~~if statement~~ `if` expression.
- `if`; `else if`; `else`.

Selekcija – IF: if-else, if kao izraz, if let

primeri/basic-concepts/src/bin/if.rs

```rust
fn main() {
    let x = 10;
    let y = 5;

    // Basic if condition
    if x > y {
        println!("x is greater than y");
    }

    // if-else condition
    if x == y {
        println!("x is equal to y");
    } else {
        println!("x is not equal to y");
    }

    // Nested if-else
    if x > y {
        println!("x is greater than y");
    } else if x < y {
        println!("x is less than y");
    } else {
        println!("x is equal to y");
```

```
    }

    // if as an expression
    // Assigns a value based on the condition
    let max = if x > y { x } else { y };
    println!("The maximum value is: {}", max);

    // Using if with multiple conditions (logical AND and OR)
    if x > 0 && y > 0 {
        println!("Both x and y are positive numbers");
    }

    if x < 0 || y < 0 {
        println!("At least one of x or y is negative");
    } else {
        println!("Neither x nor y are negative");
    }

    // Handling more complex conditions
    let z = 3;

    if x > y && y > z {
        println!("x is the largest and z is the smallest");
    } else if z > x {
        println!("z is the largest");
    } else {
        println!("Neither condition is true");
    }

    // Using if-let to work with Option types
    let some_value: Option<i32> = Some(100);

    if let Some(v) = some_value {
        println!("Found a value: {}", v);
    } else {
        println!("No value found");
    }
}
```

Izlaz

```
x is greater than y
x is not equal to y
x is greater than y
The maximum value is: 10
Both x and y are positive numbers
Neither x nor y are negative
x is the largest and z is the smallest
Found a value: 100
```

Selekcija – MATCH

- Izraz (vraća vrednost).
- Iscrpna pretraga.
- Analogan `switch` iskazu u C/C++.
- Koristi se i za *pattern matching*.

Selekcija – MATCH: vrednosti, opsezi, vezivanje

primeri/basic-concepts/src/bin/match.rs

```rust
fn main() {
    let x = 42;

    match x {
        0 => {
            println!("found zero");
        }
        // we can match against multiple values
        1 | 2 => {
            println!("found 1 or 2!");
        }
        // we can match against ranges
        3..=9 => {
            println!("found a number 3 to 9 inclusively");
        }
        // we can bind the matched number to a variable
        matched_num @ 10..=100 => {
            println!("found {} number between 10 to 100!", matched_num);
        }
        // this is the default match that must exist if not all cases are
        ↪ handled
        _ => {
            println!("found something else!");
        }
    }
}
```

Izlaz

```
found 42 number between 10 to 100!
```

Iteracija – LOOP

- Takođe *expression*.
- Izvršava deo koda beskonačno puta ili dok se eksplicitno ne zaustavi (CTRL+C):
  - `break` ili `continue`.
- Oznaka (`'naziv: loop`) omogućava da `break` i `continue` ciljaju spoljnu petlju; radi i za `while` i `for`.

Iteracija – LOOP: break, continue, vrednost petlje

primeri/basic-concepts/src/bin/loop.rs

```rust
fn main() {
    // Simple infinite loop
    let mut counter = 0;
    loop {
        counter += 1;
        println!("Infinite loop, counter: {}", counter);

        if counter >= 5 {
            break; // Exit the loop when counter reaches 5
        }
    }
```

```
    // Loop with continue, skipping an iteration
    let mut number = 0;
    loop {
        number += 1;

        if number == 3 {
            println!("Skipping number 3");
            continue; // Skip the rest of this iteration
        }

        println!("Loop, number: {}", number);

        if number >= 5 {
            break; // Exit the loop when number reaches 5
        }
    }

    // Loop with a return value (using break to return a value)
    let result = loop {
        counter += 1;

        if counter == 10 {
            break counter * 2; // Break and return counter * 2
        }
    };
    println!("Result from loop is: {}", result);

    // Nested loop with break
    let mut outer_count = 0;
    let mut inner_count = 0;
    loop {
        outer_count += 1;
        println!("Outer loop count: {}", outer_count);

        loop {
            inner_count += 1;
            println!("  Inner loop count: {}", inner_count);

            if inner_count >= 3 {
                break; // Exit inner loop when inner_count reaches 3
            }
        }

        if outer_count >= 2 {
            break; // Exit outer loop when outer_count reaches 2
        }
    }

    // Labeled loops: break the outer loop from the inner one
    'outer: loop {
        loop {
            println!("Breaking out of both loops");
            break 'outer;
        }
    }

    // Infinite loop without breaking (commented out to avoid actual
     ↪  infinite loop)
```

```
    /*
    loop {
        println!("This is an infinite loop");
    }
    */
}
```

Izlaz

```
Infinite loop, counter: 1
Infinite loop, counter: 2
Infinite loop, counter: 3
Infinite loop, counter: 4
Infinite loop, counter: 5
Loop, number: 1
Loop, number: 2
Skipping number 3
Loop, number: 4
Loop, number: 5
Result from loop is: 20
Outer loop count: 1
  Inner loop count: 1
  Inner loop count: 2
  Inner loop count: 3
Outer loop count: 2
  Inner loop count: 4
Breaking out of both loops
```

Iteracija – WHILE

- *Expression* tipa `()` (ne vraća vrednost).

Iteracija – WHILE: uslov, break, continue

primeri/basic-concepts/src/bin/while.rs

```rust
fn main() {
    // Basic while loop
    let mut count = 0;
    while count < 5 {
        println!("Basic while loop, count: {}", count);
        count += 1;
    }

    // While loop with break
    let mut number = 10;
    while number > 0 {
        println!("While loop with break, number: {}", number);
        number -= 1;

        if number == 5 {
            println!("Breaking the loop as number reached 5");
            break; // Exit the loop when number is 5
        }
    }

    // While loop with continue
    let mut skip_number = 0;
```

```
    while skip_number < 5 {
        skip_number += 1;

        if skip_number == 3 {
            println!("Skipping number 3");
            continue;
        }

        println!("While loop with continue, skip_number: {}",
         ↪  skip_number);
    }

    // While loop that checks a condition from user input (simulated here)
    let mut user_input = 0;
    while user_input != 42 {
        user_input += 7; // Simulating user input increment
        println!("While loop, simulating user input: {}", user_input);
    }
    println!("Exited loop, user_input reached 42");
}
```

Iteracija – FOR

- *Expression* tipa `()` (ne vraća vrednost).

Iteracija – FOR: opseg, kolekcije, rev, step_by

primeri/basic-concepts/src/bin/for.rs

```
fn main() {
    // For loop over a range
    for i in 0..5 {
        println!("For loop over range, i: {}", i);
    }

    // For loop over an inclusive range
    for i in 0..=5 {
        println!("For loop over inclusive range, i: {}", i);
    }

    // For loop iterating over an array
    let arr = [10, 20, 30, 40, 50];
    for element in arr.into_iter() {
        println!("For loop over array, element: {}", element);
    }
    for element in arr.iter() {
        println!("For loop over array, element: {}", element);
    }

    // For loop iterating over a vector
    let vec = vec![1, 2, 3, 4, 5];
    for element in vec.iter() {
        println!("For loop over vector, element: {}", element);
    }
    for element in vec.into_iter() {
        println!("For loop over vector, element: {}", element);
    }
```

```
    // For loop with index using enumerate
    for (index, element) in arr.iter().enumerate() {
        println!(
            "For loop with enumerate, index: {}, element: {}",
            index, element
        );
    }

    // For loop over a reversed range
    for i in (0..5).rev() {
        println!("For loop over reversed range, i: {}", i);
    }

    // For loop over a range with a step
    for i in (0..10).step_by(3) {
        println!("For loop with step_by, i: {}", i);
    }

    // For loop with break
    for i in 0..10 {
        if i == 5 {
            println!("Breaking the loop at i: {}", i);
            break; // Exit the loop when i is 5
        }
        println!("For loop, i: {}", i);
    }

    // For loop with continue
    for i in 0..5 {
        if i == 3 {
            println!("Skipping i: {}", i);
            continue; // Skip the rest of the current iteration
        }
        println!("For loop with continue, i: {}", i);
    }
}
```

Izlaz

```
For loop over range, i: 0
For loop over range, i: 1
For loop over range, i: 2
For loop over range, i: 3
For loop over range, i: 4
For loop over inclusive range, i: 0
For loop over inclusive range, i: 1
For loop over inclusive range, i: 2
For loop over inclusive range, i: 3
For loop over inclusive range, i: 4
For loop over inclusive range, i: 5
For loop over array, element: 10
For loop over array, element: 20
For loop over array, element: 30
For loop over array, element: 40
For loop over array, element: 50
For loop over array, element: 10
For loop over array, element: 20
For loop over array, element: 30
For loop over array, element: 40
```

```
For loop over array, element: 50
For loop over vector, element: 1
For loop over vector, element: 2
For loop over vector, element: 3
For loop over vector, element: 4
For loop over vector, element: 5
For loop over vector, element: 1
For loop over vector, element: 2
For loop over vector, element: 3
For loop over vector, element: 4
For loop over vector, element: 5
For loop with enumerate, index: 0, element: 10
For loop with enumerate, index: 1, element: 20
For loop with enumerate, index: 2, element: 30
For loop with enumerate, index: 3, element: 40
For loop with enumerate, index: 4, element: 50
For loop over reversed range, i: 4
For loop over reversed range, i: 3
For loop over reversed range, i: 2
For loop over reversed range, i: 1
For loop over reversed range, i: 0
For loop with step_by, i: 0
For loop with step_by, i: 3
For loop with step_by, i: 6
For loop with step_by, i: 9
For loop, i: 0
For loop, i: 1
For loop, i: 2
For loop, i: 3
For loop, i: 4
Breaking the loop at i: 5
For loop with continue, i: 0
For loop with continue, i: 1
For loop with continue, i: 2
Skipping i: 3
For loop with continue, i: 4
```

Kolekcije

Vector

- Dinamička, sekvencijalna struktura.
- Sve vrednosti istog tipa.
- Neke od operacija:
  - `push`,
  - `get`.

Vector: dodavanje, uklanjanje, iteracija

`primeri/basic-concepts/src/bin/vector.rs`

```rust
fn main() {
    // Creating a new empty vector
    let mut numbers: Vec<i32> = Vec::new();

    // Adding elements to the vector using push
    numbers.push(10);
```

```
    numbers.push(20);
    numbers.push(30);
    println!("Vector after pushing elements: {:?}", numbers);
    println!("Length: {}", numbers.len());

    // Accessing elements of the vector by index
    println!("First element: {}", numbers[0]);

    // Safely accessing elements using get (returns an Option)
    match numbers.get(3) {
        Some(value) => println!("Fourth element: {}", value),
        None => println!("No fourth element"),
    }

    // Iterating over the vector using a for loop
    for number in &numbers {
        println!("Iterating over vector, number: {}", number);
    }

    // Iterating with mutable references to modify the vector in place
    for number in &mut numbers {
        *number += 5; // Adding 5 to each element
    }
    println!("Vector after modifying each element: {:?}", numbers);

    // Removing the last element of the vector using pop
    match numbers.pop() {
        Some(value) => println!("Popped value: {}", value),
        None => println!("No value to pop"),
    }
    println!("Vector after popping an element: {:?}", numbers);

    // Inserting an element at a specific position
    numbers.insert(1, 25); // Inserting 25 at index 1
    println!("Vector after inserting 25 at index 1: {:?}", numbers);

    // Removing an element at a specific index
    let removed_element = numbers.remove(1); // Removes the element at
     ↪  index 1
    println!("Removed element: {}", removed_element);
    println!("Vector after removing element at index 1: {:?}", numbers);

    // Extending the vector by adding another vector
    let more_numbers = vec![40, 50, 60];
    numbers.extend(more_numbers);
    println!("Vector after extending with more numbers: {:?}", numbers);

    // Iterating with index using enumerate
    for (index, number) in numbers.iter().enumerate() {
        println!("Index: {}, Value: {}", index, number);
    }

    // Clearing the vector (removes all elements)
    numbers.clear();
    println!("Vector after clearing all elements: {:?}", numbers);
    println!("Is empty: {}", numbers.is_empty());
}
```

Izlaz

```
Vector after pushing elements: [10, 20, 30]
Length: 3
First element: 10
No fourth element
Iterating over vector, number: 10
Iterating over vector, number: 20
Iterating over vector, number: 30
Vector after modifying each element: [15, 25, 35]
Popped value: 35
Vector after popping an element: [15, 25]
Vector after inserting 25 at index 1: [15, 25, 25]
Removed element: 25
Vector after removing element at index 1: [15, 25]
Vector after extending with more numbers: [15, 25, 40, 50, 60]
Index: 0, Value: 15
Index: 1, Value: 25
Index: 2, Value: 40
Index: 3, Value: 50
Index: 4, Value: 60
Vector after clearing all elements: []
Is empty: true
```

HashMap

- Nelinearna, dinamička struktura.
- (Ključ, Vrednost).
- Neke od operacija:
  - `insert`,
  - `entry`,
  - `or_insert`,
  - `get`.

HashMap: ubacivanje, čitanje, `entry`

`primeri/basic-concepts/src/bin/hashmap.rs`

```rust
use std::collections::HashMap;

fn main() {
    // Creating a new empty HashMap
    let mut scores: HashMap<String, i32> = HashMap::new();

    // Inserting key-value pairs into the HashMap
    scores.insert(String::from("Alice"), 10);
    scores.insert(String::from("Bob"), 20);
    scores.insert(String::from("Charlie"), 30);
    println!("Initial scores: {:?}", scores);

    // Accessing a value by key
    let alice_score = scores.get("Alice");
    match alice_score {
        Some(score) => println!("Alice's score: {}", score),
        None => println!("No score found for Alice"),
    }

    // Iterating over the HashMap
    for (key, value) in &scores {
        println!("Player: {}, Score: {}", key, value);
```

```
    }

    // Updating the value of an existing key
    scores.insert(String::from("Alice"), 50); // Alice's score is updated
    println!("Scores after updating Alice: {:?}", scores);

    // Using entry() to conditionally insert a value if the key is not
     ↪  present
    scores.entry(String::from("David")).or_insert(40);
    println!("Scores after adding David: {:?}", scores);

    // Using entry() to modify the value if the key exists
    let alice_entry = scores.entry(String::from("Alice")).or_insert(0);
    *alice_entry += 10; // Adds 10 to Alice's score
    println!("Scores after modifying Alice using entry(): {:?}", scores);

    // Checking if a key exists
    if scores.contains_key("Bob") {
        println!("Bob is in the scores map");
    }

    // Upserting a value (insert if absent, update if present)
    scores.insert(String::from("Bob"), 25); // Bob's score is updated

    // Removing a key-value pair
    scores.remove("Charlie");
    println!("Scores after removing Charlie: {:?}", scores);

    // Clearing the HashMap (removes all entries)
    scores.clear();
    println!("Scores after clearing all entries: {:?}", scores);
}
```

Izlaz

```
Initial scores: {"Alice": 10, "Bob": 20, "Charlie": 30}
Alice's score: 10
Player: Alice, Score: 10
Player: Bob, Score: 20
Player: Charlie, Score: 30
Scores after updating Alice: {"Alice": 50, "Bob": 20, "Charlie": 30}
Scores after adding David: {"David": 40, "Alice": 50, "Bob": 20,
 ↪  "Charlie": 30}
Scores after modifying Alice using entry(): {"David": 40, "Alice": 60,
 ↪  "Bob": 20, "Charlie": 30}
Bob is in the scores map
Scores after removing Charlie: {"David": 40, "Alice": 60, "Bob": 25}
Scores after clearing all entries: {}
```

Zadaci

Zadatak 1

Napisati algoritam u Rust programskom jeziku koji pretvara temperaturu iz °F u °C.

- Zaokružiti vrednost na dve decimale.
- Kao vrednost °F koristiti vrednost sa standardnog ulaza.

Zadatak 2

Napisati algoritam u Rust programskom jeziku koji generiše prvih N Fibonačijevih brojeva.

- Vrednost N učitati sa standardnog ulaza.
- Brojeve ispisati u jednom redu, razdvojene zarezom.

Zadatak 3

Napisati algoritam u Rust programskom jeziku koji ispisuje da li je uneti broj prost.

Zadatak 4

Napisati algoritam u Rust programskom jeziku koji omogućava korisniku da unese veličinu vektora, a zatim:

- Dinamički popuniti vektor.
- Ispisati sve elemente vektora.
- Ispisati elemente vektora u obrnutom redosledu.
- Ispisati element na indeksu koji je korisnik uneo. Ako nema elementa na traženom indeksu, korisnika obavestiti o tome.
- Ispisati sve elemente koji se nalaze na indeksu koji je deljiv sa 3.
- Prebrojati koliko se elemenata nalazi na parnim, a koliko na neparnim indeksima (koristiti `HashMap`).

Izvori

Izvori

- Rust Community. “The Rust Programming Language - the Rust Programming Language.” Rust-Lang.org, 2018, doc.rust-lang.org/book/.
- Rust Team. “Rust Programming Language.” Rust-Lang.org, 2018, www.rust-lang.org/.
- Rust Community. “Tour of Rust - Let’s Go on an Adventure!” Tourofrust.com, tourofrust.com/.